Restaurant Management System (RMS)
Software Requirements Specification

Contents

1 Introduction **4**
- 1.1 Purpose . . . 4
- 1.2 Scope . . . 4
- 1.3 Definitions / Acronyms . . . 4
- 1.4 Overview . . . 4

2 Overall Description **6**
- 2.1 Product Perspective . . . 6
- 2.2 Product Functions . . . 6
- 2.3 User Classes & Characteristics . . . 6
- 2.4 Operating Environment . . . 8
- 2.5 Design Constraints . . . 8
- 2.6 Assumptions & Dependencies . . . 8

3 Functional Requirements **9**
- 3.1 Authentication . . . 9
- 3.2 Order Creation . . . 9
- 3.3 Kitchen Workflow . . . 10
- 3.4 Payment Processing . . . 10
- 3.5 Refund . . . 10
- 3.6 Inventory Management . . . 10
- 3.7 Order Cancellation . . . 11
- 3.8 Reporting . . . 11
- 3.9 Requirements Table . . . 11

4 Non-Functional Requirements **12**
- 4.1 Performance . . . 12
- 4.2 Security . . . 12
- 4.3 Reliability . . . 12
- 4.4 Scalability . . . 12
- 4.5 Usability . . . 12
- 4.6 Metrics Table . . . 12

5 Supporting Information **13**
- 5.1 ER Diagram (PlantUML) . . . 13
- 5.2 Database Schema . . . 13

6 Architecture **22**
6.1 Folder Structure 22
6.2 API Contract 22

Document Information

Document Title:	Restaurant Management System SRS
Project:	RMS (Restaurant Management System)
Version:	1.0.0
Type:	Software Requirements Specification
Status:	Draft

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the **Restaurant Management System (RMS)**, a web-based application designed to manage restaurant operations including orders, payments, inventory, kitchen workflow, tables, users, and reporting.

1.2 Scope

The Restaurant Management System will:

- Enable waiters to create and manage orders
- Allow chefs to manage kitchen workflow
- Allow cashiers to process partial/full payments and refunds
- Manage inventory automatically during order processing
- Provide reporting and analytics for management
- Support role-based access control

Business Value

- Reduces manual errors in billing
- Tracks inventory automatically
- Improves kitchen efficiency
- Enables financial reporting
- Supports audit logging

1.3 Definitions / Acronyms

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token
RMS	Restaurant Management System
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
ACID	Atomicity, Consistency, Isolation, Durability
UI	User Interface

1.4 Overview

This document includes:

- System overview
- Functional requirements
- Non-functional requirements
- Data model
- Architecture
- API contract summary

2 Overall Description

2.1 Product Perspective

The RMS is a **new web-based system** built using:

- **Backend:** Go (Clean Monolith Architecture)
- **Frontend:** TypeScript (React)
- **Database:** MongoDB
- **Deployment:** Free-tier cloud hosting

The system is modular and designed to scale to microservices in future.

2.2 Product Functions

High-level features:

- User Authentication & Role Management
- Table Management
- Menu & Category Management
- Order Management (with state machine)
- Payment Processing (partial & full)
- Refund Handling
- Inventory Management
- Reporting & Analytics
- Audit Logging

2.3 User Classes & Characteristics

1 Admin (Owner)

Full system control.
Capabilities:

- Create/manage staff accounts
- View sales analytics
- Configure tax settings
- Manage menu categories
- View inventory reports
- Access all modules

2 Manager

Operational control.

Capabilities:

- Manage tables
- Assign waiters
- Modify menu items
- View live orders
- Override billing issues
- Generate reports

3 Waiter

Front-of-house staff.

Capabilities:

- View assigned tables
- Create order
- Modify order
- Send order to kitchen
- View order status
- Request bill

4 Kitchen Staff

Back-of-house.

Capabilities:

- View live order queue
- Update order status
  - Preparing
  - Ready
- View order notes
- Mark item out-of-stock (optional)

5 Cashier

Billing desk.

Capabilities:

- View completed orders
- Generate final bill
- Process payment
- Mark order as paid
- Print receipt

2.4 Operating Environment

- **Web-based application**
- **Supported Browsers:**
  - Chrome (v100+)
  - Firefox (v100+)
  - Edge
- **Backend:**
  - Linux server
- **Database:**
  - MongoDB Atlas

2.5 Design Constraints

- Must use Go (backend)
- Must use TypeScript (frontend)
- MongoDB as primary database
- Clean Monolith architecture
- RESTful API
- JWT authentication

2.6 Assumptions & Dependencies

- Internet connection required
- MongoDB availability
- Users have modern browser
- Cloud deployment platform availability

3 Functional Requirements

3.1 Authentication

REQ-001: User Login

As a user, **I want to** log in securely **so that** I can access my authorized dashboard.
Acceptance Criteria:

- Must validate credentials
- Must generate JWT
- Must return role info
- Must reject invalid login

Workflow:

1. User enters email/password
2. Backend validates
3. JWT issued
4. User redirected to dashboard

3.2 Order Creation

REQ-002: Create Order

As a waiter, **I want to** create an order for a table **so that** customers can be served.
Acceptance:

- Table must be available
- At least 1 item required
- Total amount auto-calculated
- Table marked OCCUPIED

Workflow:

1. Select table
2. Select items
3. Confirm order
4. System stores order

3.3 Kitchen Workflow

REQ-003: Update Order Status

As a chef, **I want to** update order status **so that** workflow is tracked.
Allowed transitions: PLACED → PREPARING → READY → SERVED
Acceptance:

- Invalid transitions rejected
- Only Chef role allowed

3.4 Payment Processing

REQ-004: Process Partial Payment

As a cashier, **I want to** process payments in parts **so** customers can split bills.
Acceptance:

- Cannot exceed remaining amount
- Payment recorded
- Order status = PAID only when fully paid
- Table released after full payment

3.5 Refund

REQ-005: Refund Payment

As a cashier, **I want to** refund a payment **so that** billing mistakes can be corrected.
Acceptance:

- Cannot refund more than paid
- Payment marked REFUNDED
- Order status updated

3.6 Inventory Management

REQ-006: Deduct Inventory

As a system, **I want** inventory deducted during payment **so** stock remains accurate.
Acceptance:

- If insufficient stock → fail payment
- Transaction rollback if failure

3.7 Order Cancellation

REQ-007: Cancel Order

As a waiter/manager, **I want to** cancel orders before payment.
Acceptance:

- Cannot cancel PAID orders
- Inventory restored
- Table released

3.8 Reporting

REQ-008: View Sales Report

As a manager, **I want to** view total sales **so** I can analyze performance.
Acceptance:

- Filter by date range
- Show total sales
- Show total orders

3.9 Requirements Table

Req ID	Description	Priority	Workflow
REQ-001	User login	High	Auth Flow
REQ-002	Create order	High	Order Journey
REQ-003	Update status	High	Kitchen Flow
REQ-004	Partial payment	High	Payment Flow
REQ-005	Refund	Medium	Refund Flow
REQ-006	Inventory deduction	High	Payment Transaction
REQ-007	Cancel order	Medium	Cancellation
REQ-008	Sales report	Medium	Reporting

4 Non-Functional Requirements

4.1 Performance

- API response time ¡ 300ms (average)
- Support 1,000 concurrent users
- Page load time ¡ 2 seconds

4.2 Security

- JWT authentication
- Password hashing (bcrypt)
- Role-based access control
- HTTPS only
- Input validation & sanitization

4.3 Reliability

- 99.9% uptime target
- Database backup daily
- Transaction safety for payments

4.4 Scalability

- Handle 10,000 users/day
- Horizontally scalable backend
- Designed for future microservice migration

4.5 Usability

- Mobile responsive UI
- WCAG 2.1 AA accessibility
- Simple dashboard layout

4.6 Metrics Table

Category	**Requirement**	**Metric**
Capacity	Daily Users	10k
Storage	Initial	50GB
Compatibility	Browsers	Chrome, Firefox, Edge

5 Supporting Information

5.1 ER Diagram (PlantUML)

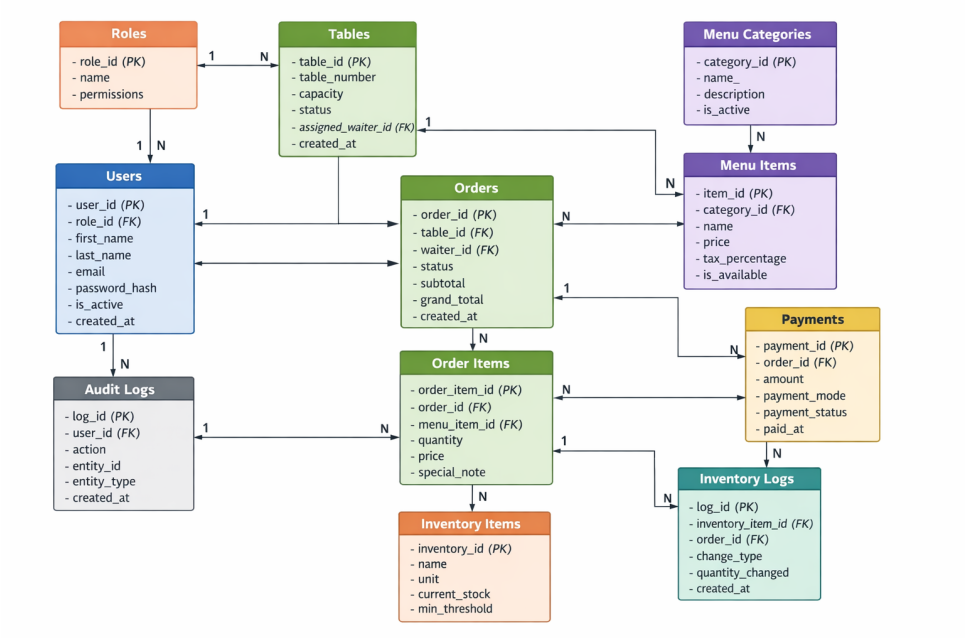

Figure 1: ER Diagram for RMS

5.2 Database Schema

1 Design Principles

Before defining schema:

We will:

- Use **ObjectId references** for relationships
- Avoid heavy nesting (orders can grow large)
- Use enum-like status fields
- Use audit fields everywhere
- Index heavily queried fields
- Prepare for future multi-branch support

2 Core Collections

```
users
roles
tables
menu_categories
menu_items
orders
```

```
order_items
payments
inventory_items
inventory_logs
settings
audit_logs
```

3 USERS Collection

Used for: Admin, Manager, Waiter, Kitchen, Cashier

Schema:

```
{
    "_id": ObjectId,
    "first_name": "Soham",
    "last_name": "Panji",
    "email": "soham@example.com",
    "password_hash": "bcrypt_hash",
    "role_id": ObjectId,
    "is_active": true,
    "phone": "9876543210",
    "last_login_at": ISODate,
    "created_at": ISODate,
    "updated_at": ISODate,
    "deleted_at": null
}
```

Indexes:

- unique: email
- index: role_id
- index: is_active

4 ROLES Collection

Keeps RBAC scalable.

```
{
    "_id": ObjectId,
    "name": "WAITER",
    "permissions": [
        "ORDER_CREATE",
        "ORDER_UPDATE",
        "VIEW_ASSIGNED_TABLE"
    ],
    "created_at": ISODate
}
```

Why separate?

- Easy permission expansion
- Future role changes without code changes

5 TABLES Collection

```
{
    "_id": ObjectId,
    "table_number": 12,
    "capacity": 4,
    "status": "AVAILABLE",
    "assigned_waiter_id": ObjectId,
    "current_order_id": ObjectId,
    "created_at": ISODate,
    "updated_at": ISODate
}
```

Status Enum:

- AVAILABLE
- OCCUPIED
- RESERVED
- CLEANING

Indexes:

- unique: table_number
- index: status
- index: assigned_waiter_id

6 MENU_CATEGORIES Collection

```
{
    "_id": ObjectId,
    "name": "Beverages",
    "description": "Drinks and juices",
    "is_active": true,
    "created_at": ISODate
}
```

Index:

- unique: name

7 MENU_ITEMS Collection

```
{
    "_id": ObjectId,
    "name": "Chicken Burger",
    "category_id": ObjectId,
    "price": 250,
    "tax_percentage": 5,
```

```
    "is_available": true,
    "preparation_time_minutes": 15,
    "image_url": "",
    "created_at": ISODate,
    "updated_at": ISODate,
    "deleted_at": null
}
```

Indexes:

- index: category_id
- index: is_available
- text index: name

8 ORDERS Collection (Core System)

We separate order_items into another collection for scalability.

ORDERS:

```
{
    "_id": ObjectId,
    "order_number": "ORD-2026-00012",
    "table_id": ObjectId,
    "waiter_id": ObjectId,
    "status": "PREPARING",
    "subtotal": 500,
    "tax_total": 25,
    "service_charge": 50,
    "grand_total": 575,
    "notes": "Customer birthday",
    "started_at": ISODate,
    "completed_at": ISODate,
    "paid_at": ISODate,
    "created_at": ISODate,
    "updated_at": ISODate,
    "deleted_at": null
}
```

Status Enum:

- CREATED
- SENT_TO_KITCHEN
- PREPARING
- READY
- SERVED
- COMPLETED
- CANCELLED

- PAID

Indexes:

- unique: order_number
- index: table_id
- index: status
- index: waiter_id
- index: created_at

9 ORDER_ITEMS Collection

Why separate?

- Large orders won't bloat order document
- Easier item-level updates
- Better performance

```
{
    "_id": ObjectId,
    "order_id": ObjectId,
    "menu_item_id": ObjectId,
    "quantity": 2,
    "price_at_order_time": 250,
    "tax_percentage": 5,
    "item_status": "PREPARING",
    "special_note": "Less spicy",
    "created_at": ISODate,
    "updated_at": ISODate
}
```

Item Status:

- PENDING
- PREPARING
- READY
- SERVED
- CANCELLED

Indexes:

- index: order_id
- index: menu_item_id
- index: item_status

PAYMENTS Collection

```
{
    "_id": ObjectId,
    "order_id": ObjectId,
    "amount": 575,
    "payment_mode": "UPI",
    "payment_status": "SUCCESS",
    "transaction_reference": "TXN123456",
    "paid_at": ISODate,
    "created_at": ISODate
}
```

Payment Modes:

- CASH
- CARD
- UPI
- WALLET

Payment Status:

- PENDING
- SUCCESS
- FAILED
- REFUNDED

Indexes:

- index: order_id
- index: payment_status
- index: paid_at

11 INVENTORY_ITEMS (Phase 2)

```
{
    "_id": ObjectId,
    "name": "Tomato",
    "unit": "kg",
    "current_stock": 25,
    "minimum_threshold": 5,
    "is_active": true,
    "created_at": ISODate
}
```

Index:

- unique: name

12 INVENTORY_LOGS

```
{
    "_id": ObjectId,
    "inventory_item_id": ObjectId,
    "change_type": "DEDUCTION",
    "quantity_changed": -2,
    "reference_order_id": ObjectId,
    "created_at": ISODate
}
```

13 SETTINGS Collection

Global configs.

```
{
    "_id": ObjectId,
    "tax_percentage": 5,
    "service_charge_percentage": 10,
    "currency": "INR",
    "restaurant_name": "Soham's Kitchen",
    "updated_at": ISODate
}
```

14 AUDIT_LOGS Collection

Track critical actions.

```
{
    "_id": ObjectId,
    "user_id": ObjectId,
    "action": "ORDER_CANCELLED",
    "entity_type": "ORDER",
    "entity_id": ObjectId,
    "metadata": {},
    "created_at": ISODate
}
```

Index:

- index: entity_type
- index: entity_id
- index: created_at

15 Relationship Map

- User → Role
- Table → Waiter
- Order → Table

- Order → Waiter
- OrderItem → Order
- OrderItem → MenuItem
- Payment → Order
- InventoryLog → InventoryItem

16 Reporting Optimization

For daily sales:

Index:

- orders.created_at
- payments.paid_at

Aggregation pipelines:

- Total revenue per day
- Top selling items
- Revenue per category

17 Soft Delete Strategy

We use:

```
deleted_at: ISODate | null
```

Never hard delete:

- Orders
- Payments
- Users

Hard delete allowed:

- Temporary inventory logs (optional)

18 Production Optimizations

1. Generate order_number using counter collection:

```
counters
{
    _id: "order",
    sequence_value: 124
}
```

2. Use transactions when:

- Payment processed
- Table status updated
- Order marked paid

MongoDB supports ACID transactions.

6 Architecture

6.1 Folder Structure

```
restaurant/
 cmd/
    server/
 internal/
    domain/
       entities/
       repositories/
    application/
       services/
    infrastructure/
       persistence/mongo/
    delivery/http/
 frontend/
    src/
    components/
 docs/
 tests/
 openapi.yaml
```

6.2 API Contract

```
openapi: 3.0.3
info:
  title: Restaurant Management API
  description: Clean Monolith Restaurant Management System
  version: 1.0.0
servers:
  - url: /api/v1
security:
  - BearerAuth: []
tags:
  - name: Auth
  - name: Users
  - name: Tables
  - name: Menu
  - name: Orders
  - name: Payments
  - name: Inventory
  - name: Reports

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

```
  schemas:
    ErrorResponse:
      type: object
      properties:
        error:
          type: object
          properties:
            code:
              type: string
            message:
              type: string
    PaginationMeta:
      type: object
      properties:
        page:
          type: integer
        limit:
          type: integer
        total:
          type: integer
    PaginatedResponse:
      type: object
      properties:
        data:
          type: array
          items:
            type: object
        meta:
          $ref: '#/components/schemas/PaginationMeta'
    User:
      type: object
      properties:
        id:
          type: string
        name:
          type: string
        email:
          type: string
        role:
          type: string
          enum: [ADMIN, MANAGER, WAITER, CHEF, CASHIER]
    Table:
      type: object
      properties:
        id:
          type: string
        number:
          type: integer
        capacity:
          type: integer
        status:
```

```
        type: string
        enum: [AVAILABLE, OCCUPIED, RESERVED]
    MenuCategory:
      type: object
      properties:
        id:
          type: string
        name:
          type: string
    MenuItem:
      type: object
      properties:
        id:
          type: string
        name:
          type: string
        category_id:
          type: string
        price:
          type: number
        available:
          type: boolean
    OrderItem:
      type: object
      properties:
        menu_item_id:
          type: string
        quantity:
          type: integer
        price:
          type: number
    Order:
      type: object
      properties:
        id:
          type: string
        table_id:
          type: string
        status:
          type: string
          enum: [PLACED, PREPARING, READY, SERVED, PAID,
             CANCELLED]
        total_amount:
          type: number
        paid_amount:
          type: number
        items:
          type: array
          items:
            $ref: '#/components/schemas/OrderItem'
    Payment:
```

```
      type: object
      properties:
        id:
          type: string
        order_id:
          type: string
        amount:
          type: number
        method:
          type: string
          enum: [CASH, CARD, UPI]
        status:
          type: string
          enum: [COMPLETED, REFUNDED]
        created_at:
          type: string
          format: date-time
    RefundRequest:
      type: object
      properties:
        amount:
          type: number
        reason:
          type: string

paths:
  /auth/login:
    post:
      tags: [Auth]
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email:
                  type: string
                password:
                  type: string
      responses:
        200:
          description: Login successful
        401:
          description: Unauthorized

  /users:
    get:
      tags: [Users]
```

```
    responses:
      200:
        description: List users
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaginatedResponse'

/tables:
  get:
    tags: [Tables]
    responses:
      200:
        description: List tables
  post:
    tags: [Tables]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [number, capacity]
            properties:
              number:
                type: integer
              capacity:
                type: integer
    responses:
      201:
        description: Table created

/menu/categories:
  get:
    tags: [Menu]
    responses:
      200:
        description: List categories
  post:
    tags: [Menu]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [name]
            properties:
              name:
                type: string
    responses:
```

```
      201:
        description: Category created

/menu/items:
  get:
    tags: [Menu]
    responses:
      200:
        description: List menu items
  post:
    tags: [Menu]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [name, category_id, price]
            properties:
              name:
                type: string
              category_id:
                type: string
              price:
                type: number
              available:
                type: boolean
    responses:
      201:
        description: Menu item created

/orders:
  post:
    tags: [Orders]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [table_id, items]
            properties:
              table_id:
                type: string
              items:
                type: array
                items:
                  type: object
                  required: [menu_item_id, quantity]
                  properties:
                    menu_item_id:
```

```
                        type: string
                      quantity:
                        type: integer
      responses:
        201:
          description: Order created
    get:
      tags: [Orders]
      responses:
        200:
          description: List orders

  /orders/{id}:
    get:
      tags: [Orders]
      parameters:
        - in: path
          name: id
          required: true
          schema:
            type: string
      responses:
        200:
          description: Order details
    delete:
      tags: [Orders]
      responses:
        200:
          description: Order cancelled

  /orders/{id}/status:
    patch:
      tags: [Orders]
      parameters:
        - in: path
          name: id
          required: true
          schema:
            type: string
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [status]
              properties:
                status:
                  type: string
                  enum: [PREPARING, READY, SERVED]
      responses:
```

```
      200:
        description: Status updated

/orders/{id}/payments:
  post:
    tags: [Payments]
    parameters:
      - in: path
        name: id
        required: true
        schema:
          type: string
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [amount, method]
            properties:
              amount:
                type: number
              method:
                type: string
                enum: [CASH, CARD, UPI]
    responses:
      201:
        description: Payment processed

/payments/{id}/refund:
  post:
    tags: [Payments]
    parameters:
      - in: path
        name: id
        required: true
        schema:
          type: string
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/RefundRequest'
    responses:
      200:
        description: Refund processed

/inventory:
  get:
    tags: [Inventory]
```

```
    responses:
      200:
        description: Inventory list
  post:
    tags: [Inventory]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [menu_item_id, quantity]
            properties:
              menu_item_id:
                type: string
              quantity:
                type: integer
              threshold:
                type: integer
    responses:
      201:
        description: Inventory created

/reports/sales:
  get:
    tags: [Reports]
    parameters:
      - in: query
        name: from
        schema:
          type: string
          format: date
      - in: query
        name: to
        schema:
          type: string
          format: date
    responses:
      200:
        description: Sales report
```